

Тестовое задание для Backend-разработчика

Важно: ссылку на выполненное тестовое необходимо указать в резюме в блок «О себе».

Срок выполнения: 3 дня.

Формат сдачи: ссылка на репозиторий.

Бизнес-кейс: ИИ-помощник по складскому учёту и планированию закупок spa-оператора

Проект: Демонстрационный MVP встроенного ИИ-помощника, который на основе данных о движении товаров, текущих остатках, закупках, ценах, сроках поставки и сроках годности помогает планировать закупочную деятельность, контролировать состояние склада, прогнозировать потребность в товарах, рассчитывать рекомендуемый объём закупки, формировать бюджет закупок на месяц, квартал, полугодие и год и выявлять риски дефицита, излишков и списания.

Проблема: информация о закупках, расходе, остатках, ценах и сроках годности находится в разных источниках и требует ручного анализа. Управляющему сложно оперативно определить, какие позиции необходимо заказать, какие товары закончатся до следующей поставки, когда разместить заказ, какой объём закупить, как закупки повлияют на бюджет и какие партии необходимо использовать в первую очередь.

Как процесс работает сейчас (AS IS)

Сотрудники получают данные об остатках, расходе и закупках из нескольких источников. Остатки и потребность анализируются вручную, прогноз закупки формируется на основе опыта сотрудника и простой оценки среднего расхода, бюджет считается отдельно.

В расчёте могут не учитываться открытые заказы, сезонность, рост цен, плановая загрузка и сроки годности. Риски дефицита и списания выявляются несистемно. Решение о закупке принимает сотрудник, но история расчёта и его обоснование могут не сохраняться.

Как процесс должен работать (TO BE)

- Пользователь открывает MVP и выбирает объект, период и категорию либо задаёт вопрос в чате.
- Система получает данные об остатках, расходе, закупках, ценах, поставках и сроках годности.
- Расчётный модуль прогнозирует потребность и проверяет точку заказа.
- AI-помощник формирует список рисков и рекомендуемый план закупок.
- Пользователь видит объём, стоимость, дату заказа, бюджет, объяснение расчёта и исходные показатели.
- Пользователь может изменить период, бюджетный лимит или другие параметры и пересчитать результат.
- Решение может быть экспортировано или передано в следующий этап развития ABS.
- Автоматическое размещение заказа в MVP не выполняется.

Ключевые сценарии использования (User Stories)

- Как управляющий, я хочу узнать, какие позиции нужно заказать в ближайшие 14 дней, чтобы не допустить остановки процедур из-за отсутствия материалов.
- Как управляющий, я хочу рассчитать объём и стоимость закупки конкретного товара на выбранный период, чтобы сформировать заказ и бюджет.
- Как управляющий, я хочу видеть товары с риском дефицита, чтобы расставить приоритеты.
- Как управляющий, я хочу видеть партии с приближающимся сроком годности, чтобы использовать их в первую очередь или предотвратить списание.

- Как разработчик интерфейса и AI-модуля, я хочу получать через API остатки, движение товара и результат расчёта потребности с обоснованием, чтобы показать пользователю числа и объяснение к ним.

Тестовое задание

Задача: реализовать REST API сервис складского учёта и расчёта потребности в закупке с хранением данных в PostgreSQL.

- Реализовать эндпоинт **POST /api/movements**: принимает запись движения товара (дата, SKU, объект, тип операции receipt / consume / writeoff / return / correction, количество, партия, номер документа), сохраняет её в БД и возвращает идентификатор записи и пересчитанный остаток по позиции и объекту.
- Реализовать **GET /api/movements** — список движений с фильтрами по SKU, объекту, типу операции и периоду и с пагинацией (limit, offset, total).
- Реализовать **GET /api/stock** — текущие остатки по позициям и объектам. Остаток вычисляется из движений и партий, а не хранится отдельным изменяемым полем. В ответе: остаток, средний расход в день за 90 дней, запас в днях и ближайший срок годности.
- Реализовать **GET /api/stock/{sku}** — детализация по позиции: остатки по объектам и по партиям со сроками годности, ценами поступления и номерами накладных.
- Реализовать **POST /api/forecast** — расчёт потребности: на вход SKU, объект, горизонт в днях или месяцах и страховой запас в днях; на выход прогнозный расход, текущий остаток, поставки в пути, страховой запас, точка заказа, рекомендуемый объём закупки, ориентировочная стоимость, рекомендуемая дата заказа и блок explanation с исходными данными, формулами и допущениями. Рекомендуемый объём округляется вверх до кратности упаковки и минимальной партии поставщика.
- Реализовать **GET /api/alerts** — предупреждения по складу: риск дефицита (запаса меньше, чем срок поставки), приближение срока годности партии и отсутствие движения по позиции. У каждого предупреждения — уровень важности и показатели, на которых оно построено.
- Списание вести по FEFO: расход списывается с партии с ближайшим сроком годности, партия с истёкшим сроком не может быть источником расхода и закрывается отдельной операцией writeoff.
- Добавить проверки: неизвестный SKU или объект — 404; количество меньше или равно нулю для receipt и consume — 422; расход больше доступного остатка — 422 с указанием доступного количества; дата операции в будущем — 422; повторная загрузка того же номера документа — 409.
- Расчётную логику (средний расход, точка заказа, рекомендуемый объём, стоимость) вынести из слоя API в отдельный модуль: числа должны считаться детерминированными функциями и проверяться тестами без поднятия HTTP.
- Покрыть тестами через pytest списание по FEFO, расчёт остатка из движений, округление рекомендуемого объёма до упаковки и минимальной партии, расчёт точки заказа и отказ при расходе больше остатка — минимум 5 тест-кейсов.
- Весь проект должен подниматься одной командой docker compose up — предоставь Dockerfile для сервиса и docker-compose.yml с сервисом PostgreSQL, а также загрузку стартовых данных (справочник товаров, партии, движения) скриптом или фикстурами, чтобы API можно было проверить сразу после запуска.
- Репозиторий должен содержать README.md: как запустить, как запустить тесты, описание переменных окружения из .env.example.

Формат ответа POST /api/forecast

```
{  
  "sku": "OIL-001",
```

```

"name": "Массажное масло базовое (миндаль)",
"unit": "л",
"location": "MS-01",
"period": { "from": "2026-10-01", "to": "2026-12-31", "days": 92 },
"avg_daily_consumption": 1.362,
"forecast_demand": 125.3,
"current_stock": 50.39,
"incoming_qty": 20.0,
"safety_stock": 19.07,
"reorder_point": 28.6,
"recommended_purchase_qty": 75.0,
"unit_price": 1259.05,
"estimated_cost": 94428.75,
"recommended_order_date": "2026-10-25",
"stockout_date": "2026-11-01",
"explanation": {
  "data_used": [
    "Движение товара за последние 90 дней",
    "Остатки по партиям на 15.09.2026",
    "Открытые заказы к поставке в периоде"
  ],
  "formulas": [
    "средний расход/день = расход за 90 дн. ÷ 90",
    "прогноз = средний расход/день × дней в периоде",
    "объём = прогноз + страховой запас – остаток – поставки в пути"
  ],
  "assumptions": ["Цена принята на уровне последней закупки."],
  "as_of": "2026-09-15"
},
"warnings": [
  { "level": "warning", "message": "Остаток ниже точки заказа" }
]
}

```

Требования к реализации: конфигурация через .env, корректные HTTP-статусы (400, 404, 409, 422), валидация через Pydantic, миграции через Alembic, расчётный модуль отделён от слоя API.

Технические требования

Стек: Python 3.11, FastAPI, PostgreSQL, SQLAlchemy.

Управление зависимостями — на выбор (pip + requirements.txt, uv, poetry и т. д.).

Что должно быть в README

- Как запустить проект и как запустить тесты.

- Какие технологии использованы и почему.
- Архитектура решения: где расчёты, где доступ к данным, как считается остаток.
- Описание переменных окружения из `.env.example`.

Мотивация участия в проекте

- Почему тебе интересен этот проект?
- Как ты видишь свою роль в команде, которая создаёт продукт?
- Сколько времени в неделю ты готов(а) уделять проекту и в течение какого периода?